

Capítulo 1

Metodología

El propósito de este capítulo es reunir los fundamentos que sostienen el caso práctico. La metodología se organiza en tres bloques encadenados. El **bloque de text mining** convierte cada texto en datos estructurados sobre los que un algoritmo puede calcular. El **bloque de procesamiento de datos** construye, a partir de esa representación, las variables con poder predictivo y reduce su dimensión. El **bloque de modelización analítica** ajusta los clasificadores sobre esas variables, los valida y compara con distintas métricas. La Figura 1.1 resume el flujo. Cada bloque se detalla a continuación en su forma general. Su aplicación concreta, con los datos, las variables y las cifras del problema, se reserva para el capítulo siguiente.

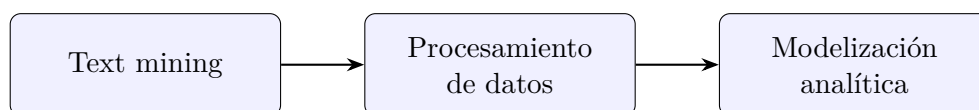


Figura 1.1: Los tres bloques de la metodología y el orden en que se aplican.

1.1. Bloque de text mining

El dato de partida es texto, pero los clasificadores que usaremos operan sobre vectores de números reales. La minería de textos (*text mining*) [1] es el conjunto de técnicas que salva esa distancia: transforma un texto en una representación numérica sobre la que un modelo pueda calcular. Este bloque recorre las piezas de esa transformación en el orden en que se aplican, de las unidades mínimas del texto hasta la matriz de números que las resume.

Tokenización. El primer paso divide el texto en unidades mínimas llamadas *tokens*, normalmente las palabras. Una tokenización simple extrae las secuencias alfanuméricas máximas y descarta los signos de puntuación, de modo que la frase queda reducida a una lista de palabras. La frase “El hotel tiene WiFi gratis.” produce los cinco tokens ⟨el, hotel, tiene, WiFi, gratis⟩. Sobre esa lista actúan los pasos siguientes.

Normalización. Distintas escrituras de una misma palabra deben tratarse como una sola. La normalización reduce esas variantes a una forma común, y agrupa varias operaciones independientes.

- **Paso a minúsculas.** Convierte todo el texto a minúsculas para que “WiFi” y “wifi” no cuenten como términos distintos.

- **Eliminación de *stopwords*.** Descarta las palabras vacías, aquellas muy frecuentes que apenas aportan significado (artículos, preposiciones, conjunciones como “de”, “el”, “y”).
- ***Stemming*.** Recorta cada palabra a su raíz aplicando reglas de sufijos, sin consultar un diccionario. Así “jugando”, “jugador” y “jugaba” se reducen todas a “jug”. Es rápido, pero la raíz que produce no siempre es una palabra real.
- **Lematización.** Lleva cada palabra a su forma de diccionario (su *lema*) usando su categoría gramatical: “jugando” pasa a “jugar”, “mejores” a “bueno”. Es más precisa que el *stemming* y más costosa, porque necesita analizar la palabra en su contexto.

Bolsa de palabras. La representación más básica de un texto es la *bolsa de palabras* [16]. Se fija un vocabulario de V términos, uno por cada palabra distinta de la colección, y cada texto se convierte en una fila de una matriz de documentos por tokens: la celda correspondiente cuenta cuántas veces aparece ese término en ese documento, sin atender al orden en que aparecen. Con los documentos “free wifi hotel”, “wifi hotel breakfast” y “cheap hotel”, el vocabulario tiene cinco términos y la matriz de frecuencias es:

	free	wifi	hotel	breakfast	cheap
d_1 : free wifi hotel	1	1	1	0	0
d_2 : wifi hotel breakfast	0	1	1	1	0
d_3 : cheap hotel	0	0	1	0	1

n -gramas. La bolsa de palabras pierde el orden, y con él parte del significado: “no funciona” dice lo contrario que “funciona”. Los n -gramas recuperan algo de ese orden tomando como unidad grupos de n tokens consecutivos en vez de palabras sueltas. Un unigrama es una palabra aislada; un bigrama, dos seguidas (“wifi gratis”); un trigramas, tres. Retener bigramas captura combinaciones que la palabra aislada no distingue, pero cada nivel de n -grama multiplica el tamaño del vocabulario, así que en la práctica el n se mantiene bajo.

1.2. Bloque de procesamiento de datos

Con el texto ya troceado, este bloque construye las variables numéricas que alimentan al modelo. La tarea es doble: pasar de simples conteos de palabras a pesos que reflejen lo informativa que es cada una, y reducir la enorme dimensión de esa representación a un espacio manejable que conserve el significado.

Ponderación TF-IDF. La matriz de frecuencias da el mismo peso a una palabra vacía muy repetida que a un término informativo raro. El esquema **TF-IDF** (*term frequency-inverse document frequency*) corrige ese sesgo ponderando cada término por su rareza en la colección [20]: pesa mucho lo específico y poco lo frecuente. Sea una colección de N documentos; para un término t , denotamos por $\text{tf}(t, d)$ el número de veces que t aparece en el documento d y por $\text{df}(t)$ el número de documentos que contienen t . El peso de t en d es el producto de dos factores,

$$w_{t,d} = \underbrace{\text{tf}(t, d)}_{\text{frecuencia en el documento}} \cdot \underbrace{\log \frac{N}{\text{df}(t)}}_{\text{rareza en la colección}}. \quad (1.1)$$

El primer factor, la frecuencia de término, sube con las veces que t aparece en el documento; el segundo, la frecuencia inversa de documento, lo penaliza si además aparece en muchos documentos. La intuición del segundo factor es una medida de sorpresa: un término presente en casi todos los documentos tiene $df(t) \approx N$, su logaritmo tiende a cero y su peso se anula, porque su presencia no aporta información; uno raro tiene $df(t)$ pequeño y su logaritmo crece.

Sobre el ejemplo de la bolsa de palabras, con $N = 3$ y tomando logaritmo natural, el factor de rareza de cada término es:

término	$df(t)$	$\ln(N/df(t))$
hotel	3	$\ln(3/3) = 0$
wifi	2	$\ln(3/2) \approx 0,41$
free, breakfast, cheap	1	$\ln(3/1) \approx 1,10$

La palabra “hotel”, presente en los tres documentos, recibe peso cero: es común y no discrimina entre documentos. Las que solo aparecen en uno reciben el peso máximo, porque son las que lo caracterizan frente al resto. Así, la fila de d_1 deja de ser $(1, 1, 1, 0, 0)$ y pasa a $(1, 10, 0, 41, 0, 0, 0)$: el peso se concentra en “free” y “hotel” se anula.

Vectores TF-IDF. Aplicada a toda la colección, la ponderación convierte cada documento en un *vector TF-IDF* de $\mathbb{R}^V$, una fila con un peso por cada término del vocabulario. Apiladas, esas filas forman la matriz TF-IDF $M \in \mathbb{R}^{N \times V}$, con un documento por fila y un término por columna. Es la representación numérica del texto sobre la que trabajan tanto las medidas de parecido entre documentos como los modelos.

1.3. Bloque de modelización analítica

Con las variables ya construidas, este bloque las convierte en un clasificador. Un modelo de aprendizaje automático es un programa que aprende a reconocer patrones a partir de ejemplos [18]: se le muestra un conjunto de datos ya resueltos y ajusta sus parámetros para reproducirlos; una vez ajustado, puede predecir sobre datos nuevos, no empleados en el entrenamiento. Según lo que se conozca de los ejemplos se distinguen dos grandes tipos. En el **aprendizaje supervisado** los ejemplos vienen etiquetados con la respuesta correcta y el modelo aprende a predecirla; es el que usamos aquí. En el **aprendizaje no supervisado** no hay etiqueta y el modelo agrupa los datos por semejanza; queda fuera de este trabajo.

Cada ejemplo se describe por unas medidas y lleva asociada una respuesta: las medidas son las *características* y la respuesta es la *clase*. Una característica es cada magnitud que se mide de un ejemplo, y el vector de características $x = (x_1, \dots, x_d) \in \mathbb{R}^d$ las reúne todas; la clase y es la respuesta asociada, y en clasificación binaria toma dos valores, $y \in \{0, 1\}$. Con n ejemplos, la base de datos es el conjunto $\{(x_i, y_i)\}_{i=1}^n$, que apilado por filas forma la matriz de características $X \in \mathbb{R}^{n \times d}$ y el vector de etiquetas $y \in \{0, 1\}^n$. El objetivo es estimar una función $f: \mathbb{R}^d \rightarrow \{0, 1\}$ que, dado un x nuevo, prediga su clase. Lo que importa es que **generalice**: que acierte sobre datos nuevos, no solo sobre los que aprendió. El obstáculo es el **sobreajuste**, un modelo con demasiada capacidad memoriza el ruido del entrenamiento y falla fuera de él, y se entiende por el compromiso **sesgo-varianza** [12]: un modelo demasiado rígido tiene sesgo alto y no captura la estructura de los datos, uno demasiado flexible tiene varianza alta y se pliega a cada fluctuación; el buen rendimiento

está en el punto intermedio. Ningún modelo es el mejor para todo problema [24], por lo que en la práctica se entrenan varios y se compara su rendimiento.

Ese proceso sigue un flujo ordenado de fases. Existen metodologías que lo estructuran. En nuestro caso adoptamos **SEMMA** [2], propuesta por el SAS Institute e implementada en la herramienta SAS Enterprise Miner, que organiza la minería de datos en cinco fases:

- **Exploración** (*Explore*): describir los datos, sus distribuciones, la prevalencia de cada clase y las relaciones entre variables.
- **Muestreo** (*Sample*): fijar los conjuntos de entrenamiento, validación y test, y equilibrar las clases si el reparto es muy desigual.
- **Modificación** (*Modify*): depurar y preparar las variables (valores ausentes, atípicos) y seleccionar las informativas.
- **Modelización** (*Model*): ajustar los modelos y sus hiperparámetros.
- **Valoración** (*Assess*): medir el rendimiento de los modelos ajustados con métricas.

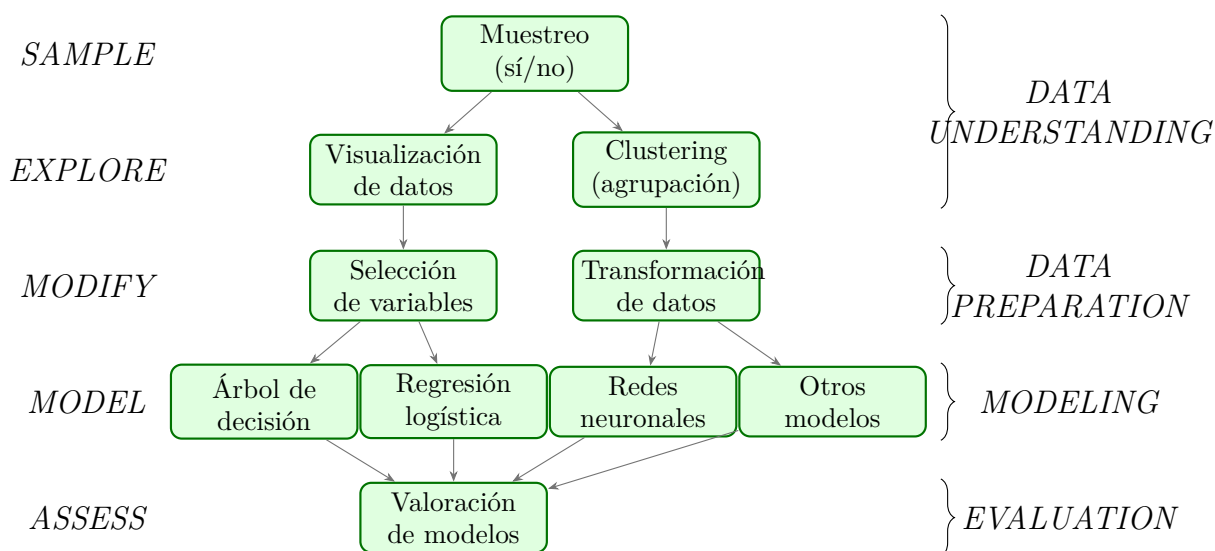


Figura 1.2: Esquema general de la organización de la minería de datos según SEMMA

Las cinco fases se desarrollan a continuación.

1.3.1. Análisis exploratorio

Antes de modelar es necesario entender los datos. El **análisis descriptivo** caracteriza cada variable y la etiqueta: su distribución, su rango, sus valores atípicos y las relaciones que guardan entre sí y con la clase. De este análisis salen las decisiones de las fases siguientes y afloran los problemas que condicionan el resto del trabajo. Los descriptivos se presentan de dos formas complementarias, y la elección depende de si interesa el resumen exacto o la forma de la distribución.

Tablas. Resumen cada variable con estadísticos numéricos: su media, su desviación típica, su mínimo y su máximo, y los cuartiles que marcan cómo se reparten los valores. Son la forma compacta y exacta de leer una variable, y sirven también para contar las categorías de una variable discreta.

Gráficos. Muestran la forma de la distribución, que una tabla de estadísticos no deja ver. Distinguimos dos usos:

- **Histograma univariante.** Reparte los valores de una variable en intervalos y dibuja una barra por intervalo con la frecuencia de cada uno. Permite ver dónde se concentran los datos, si la distribución es simétrica o sesgada, y si hay valores alejados del resto.
- **Histograma cruzado con la clase.** Superpone, sobre los mismos intervalos, la distribución de la variable en cada clase (aquí, respuestas limpias frente a alucinadas). Cuando las dos distribuciones se separan, la variable discrimina y es candidata a predictor útil; cuando se solapan por completo, la variable apenas distingue las clases.

Prevalencia y desbalance. El problema más determinante que revela la exploración en clasificación es el **desbalance** entre clases. La *prevalencia* de una clase es la fracción de ejemplos que le pertenecen; cuando una clase es mucho menos frecuente que la otra, el aprendizaje se sesga hacia la mayoritaria, y un modelo que se limite a predecirla siempre acierta en apariencia sin detectar un solo caso de la minoritaria. Si una clase supone el 90 % de los ejemplos, predecir siempre la mayoritaria alcanza un 90 % de aciertos sin ninguna utilidad. Reconocer el desbalance a tiempo orienta las tres decisiones posteriores: el muestreo con que se equilibra la base, el umbral con que se decide y las métricas con que se juzga.

1.3.2. Muestreo

La fase de **muestreo** decide con qué datos se entrena y se evalúa el modelo. Comprende tres cuestiones: si conviene trabajar con una muestra, cómo repartir los datos y cómo tratar el desbalance entre clases.

Tomar una muestra. Cuando la base es demasiado grande para ajustar los modelos en un tiempo razonable, se trabaja con una muestra aleatoria representativa en lugar de con todos los datos, sacrificando algo de información a cambio de un coste asumible.

Partición de los datos. Para estimar el rendimiento sobre datos no vistos, los datos se reparten en subconjuntos disjuntos:

- **Entrenamiento:** el modelo ajusta sobre él sus parámetros.
- **Validación:** sirve para elegir los hiperparámetros sin contaminar la estimación final.
- **Test:** se reserva hasta el final y se usa una sola vez para medir el rendimiento.

Cuando los datos escasean, apartar un bloque fijo de validación desperdicia ejemplos; en su lugar se recurre a la validación cruzada que describiremos en la modelización, que rota el papel de validación entre varias particiones del entrenamiento. En ese caso basta con separar entrenamiento y test.

Balanceo de clases. Cuando el reparto de clases es muy desigual, conviene corregirlo antes de entrenar. El **remuestreo** reequilibra la base de entrenamiento antes de ajustar el modelo, de tres maneras:

- **Sobremuestreo** (*oversampling*): añade ejemplos de la clase minoritaria hasta igualar a la mayoritaria. En su forma más simple los duplica al azar, con el riesgo de que el modelo memorice esos ejemplos repetidos.
- **SMOTE** (*Synthetic Minority Over-sampling Technique*): en lugar de duplicar, crea ejemplos *sintéticos* de la clase minoritaria interpolando entre cada ejemplo y sus vecinos más cercanos, con lo que rellena la frontera sin repetir puntos [6].
- **Submuestreo** (*undersampling*): descarta ejemplos de la clase mayoritaria hasta igualar a la minoritaria. Equilibra sin inventar datos, pero desperdicia información.

El remuestreo no es la única palanca frente al desbalance: ponderar las clases en la función de pérdida (penalizar más los errores sobre la minoritaria) o ajustar el umbral de decisión consiguen un efecto parecido sin modificar los datos.

1.3.3. Manipulación de datos

La fase de **modificación** prepara las variables antes de modelar. Depura los datos, corrigiendo los valores ausentes y los atípicos, y reduce la dimensión quedándose con las variables informativas. La construcción de las variables a partir del texto, la ingeniería de características, se describe en el capítulo práctico; aquí tratamos la teoría de la depuración y la selección.

Valores ausentes. Un valor ausente (*missing*) es una casilla sin dato: la variable no se midió, no se registró o no aplica a ese ejemplo. Importan porque muchos modelos no admiten valores ausentes, y porque la razón de la ausencia puede sesgar el análisis. Se distinguen tres mecanismos según de qué dependa que un dato falte [17]:

- **Ausentes completamente al azar** (MCAR): la ausencia no guarda relación con ningún valor, observado o no. Se pueden ignorar sin sesgo.
- **Ausentes al azar** (MAR): la ausencia depende de otras variables observadas, pero no del valor que falta. Se pueden imputar a partir de las variables conocidas.
- **Ausentes no al azar** (MNAR): la ausencia depende del propio valor que falta. Son los más problemáticos, porque ignorarlos introduce sesgo.

Ante un valor ausente caben dos vías: descartar el ejemplo, lo que desperdicia datos, o *imputarlo*, es decir, rellenarlo con un valor plausible. La imputación más simple sustituye el hueco por la media o la mediana de la variable; otras estiman el valor faltante a partir de las demás variables del ejemplo.

Valores atípicos. Un valor atípico (*outlier*) es una observación que se aparta mucho del grueso de los datos, ya por un error de medida, ya por un caso extremo real. Se detectan por reglas sobre la dispersión: un criterio habitual marca como atípico todo valor que caiga fuera del rango $[Q_1 - 1,5 \text{ RIC}, Q_3 + 1,5 \text{ RIC}]$, donde Q_1 y Q_3 son el primer y tercer cuartil y $\text{RIC} = Q_3 - Q_1$ el rango intercuartílico [23]. Los atípicos afectan sobre todo a los modelos que dependen de distancias o de sumas de cuadrados, la regresión lineal y logística o el k vecinos más cercanos, donde un solo punto extremo puede sesgar el ajuste; los modelos basados en árboles, que parten por umbrales, son mucho más robustos frente a ellos. Detectado un atípico, se corrige si es un error, se acota a un valor límite (*winsorización*) o se mantiene sin cambios cuando es un caso real informativo.

Selección de variables. Con muchas variables conviene conservar las informativas y descartar las redundantes, tanto para mejorar el modelo como para entenderlo [13]. Los métodos se agrupan en tres familias:

- **Filtro.** Puntúan cada variable por una medida estadística de su relación con la etiqueta, al margen del modelo. Son rápidos, pero ignoran las interacciones y la redundancia entre variables, así que pueden quedarse con dos variables casi idénticas.
- **Envolventes** (*wrapper*). Usan el propio modelo como juez: prueban subconjuntos de variables y seleccionan el que mejor valida. Capturan interacciones, a mayor coste.
- **Embebidos.** Seleccionan durante el propio ajuste del modelo, sin una fase aparte. La regularización L1 de la regresión logística, que lleva a cero los coeficientes de las variables irrelevantes, es un ejemplo.

La **eliminación recursiva de características** (RFE) es un método envolvente muy usado [14]: entrena con todas las variables, elimina la menos importante, reentrena y repite, lo que produce un orden de importancia. El número final de variables se fija por el *método del codo*: se representa el rendimiento frente al número de variables y se corta en el punto a partir del cual añadir más ya no aporta, donde la curva se dobla.

1.3.4. Modelización

Entrenar un clasificador es ajustar sus *parámetros* para que reproduzca las etiquetas del entrenamiento, minimizando una función de pérdida. Los modelos concretos se desarrollan más abajo; queda antes un paso común a todos: fijar sus *hiperparámetros*, las opciones que no se aprenden de los datos (la profundidad de un árbol, el número de vecinos) y que hay que decidir de antemano.

Validación cruzada. Medir el rendimiento sobre los mismos datos con que se entrena infla el resultado, porque el modelo ya los conoce. Reservar una parte para test lo evita, pero entonces el resultado depende de qué ejemplos quedaron en esa partición. La **validación cruzada** de k iteraciones elimina esa dependencia [22]: parte el entrenamiento en k bloques, entrena con $k - 1$ y valida con el restante, y repite rotando el bloque de validación (Figura 1.3); el rendimiento final es el promedio de las k rondas, que ya no depende de una partición concreta.

Esta partición debe respetar la estructura de los datos. Cuando varios ejemplos comparten una misma unidad, aquí un mismo documento fuente del que proceden varias respuestas, esa unidad tiene que caer entera en un solo bloque. Si no, el modelo encuentra en validación una respuesta casi idéntica a otra vista en entrenamiento y el resultado se infla por esa filtración. La variante que lo garantiza es el **GroupKFold**, que agrupa por la unidad e impide que un documento aparezca a la vez en entrenamiento y validación.

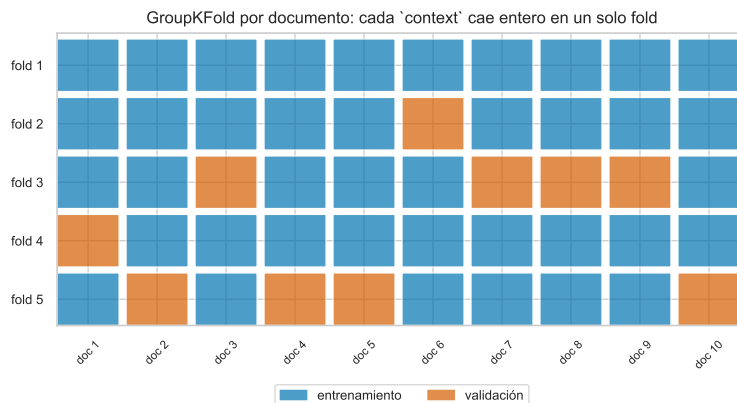


Figura 1.3: GroupKFold por documento: cada fuente cae entera en un único bloque, de modo que ninguna aparece a la vez en entrenamiento (azul) y validación (naranja).

Búsqueda en rejilla. Casi todos los modelos tienen hiperparámetros, y distintas combinaciones dan resultados distintos: no es lo mismo tomar los 3 vecinos más cercanos con distancia euclídea que los 11 con distancia de Manhattan. La **búsqueda en rejilla** (*grid search*) los ajusta probando un conjunto de combinaciones, evaluando cada una por validación cruzada y conservando la mejor, sin emplear el conjunto de test (Figura 1.4). Su coste crece con el producto de los valores de cada hiperparámetro por las iteraciones de la

validación: optimizar tres hiperparámetros con tres valores cada uno y validación cruzada de cuatro iteraciones ya son $3 \cdot 3 \cdot 3 \cdot 4 = 108$ ajustes. Para modelos clásicos ese coste es asumible en un ordenador corriente.

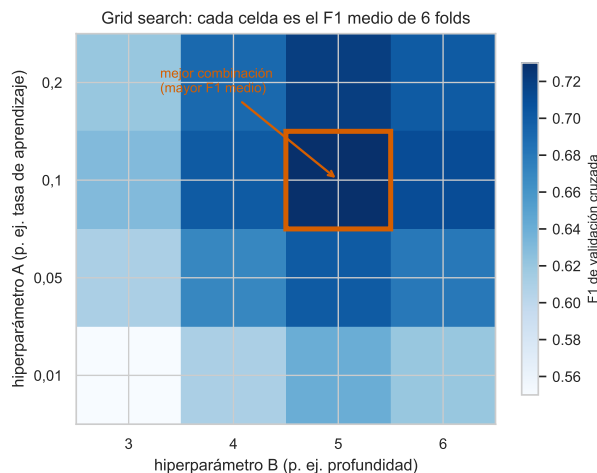


Figura 1.4: Búsqueda en rejilla: cada celda es el rendimiento medio de una combinación de hiperparámetros en validación cruzada; se conserva la mejor.

Fijado el protocolo, quedan los modelos empleados. Los presentamos en orden de complejidad creciente: la regresión logística como referencia lineal, los vecinos más cercanos y la familia de los árboles de decisión, sobre la que se construyen el bosque aleatorio y el gradient boosting.

Regresión logística

La regresión logística [9] es el clasificador lineal por excelencia y sirve de **modelo base**: la referencia simple e interpretable contra la que se miden los demás. La idea es modelar la probabilidad de la clase positiva como una combinación lineal de las características, pero acotada al intervalo $(0, 1)$ para que pueda leerse como probabilidad. El acotado se hace mediante la función *sigmoide*:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = \beta_0 + \beta^\top x, \quad p(y = 1 \mid x) = \sigma(z). \quad (1.2)$$

Aquí $\beta_0 \in \mathbb{R}$ es el término independiente y $\beta \in \mathbb{R}^d$ el vector de coeficientes, uno por característica. La sigmoide (Figura 1.5) comprime cualquier valor real a $(0, 1)$: valores muy negativos de z dan probabilidad cercana a 0, muy positivos cercana a 1, y $z = 0$ da exactamente 0,5, que marca la frontera de decisión.

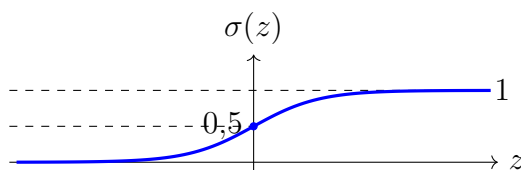


Figura 1.5: Función sigmoide: transforma la combinación lineal z en una probabilidad entre 0 y 1.

La sigmoide admite una lectura muy útil si se despeja. Partiendo de $p = \sigma(z) = 1/(1 + e^{-z})$ se tiene $1 - p = e^{-z}/(1 + e^{-z})$, y dividiendo,

$$\frac{p}{1-p} = e^z \implies \log \frac{p}{1-p} = z = \beta_0 + \beta^\top x. \quad (1.3)$$

La combinación lineal es el logaritmo de la razón entre la probabilidad de la clase positiva y la de la negativa, el *log-odds*. Esta forma da a los coeficientes una lectura directa, y es la mayor virtud del modelo: cada β_i es cuánto varía el log-odds de la clase positiva al aumentar una unidad la característica i , con las demás fijas. Su signo indica el sentido (un β_i positivo inclina la predicción hacia la clase positiva) y su magnitud, la fuerza.

Queda estimar los coeficientes β a partir del entrenamiento. El criterio es la **máxima verosimilitud**: elegir los β que hacen más probables las etiquetas observadas. Con las predicciones abreviadas $p_i = \sigma(\beta_0 + \beta^\top x_i)$ y suponiendo los ejemplos independientes, la verosimilitud de la base de datos es el producto de las probabilidades que el modelo asigna a cada etiqueta real,

$$L(\beta) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i}, \quad (1.4)$$

donde el exponente selecciona el factor que toca: p_i si $y_i = 1$ y $1 - p_i$ si $y_i = 0$. Maximizar un producto de muchos factores es poco práctico, por lo que se toman logaritmos, que convierten el producto en suma y no alteran la posición del máximo, por ser el logaritmo creciente. Cambiando el signo para pasar de maximizar a minimizar y dividiendo por n para promediar, se obtiene la *entropía cruzada*, la función de pérdida que castiga predecir con confianza la clase equivocada:

$$\mathcal{L}(\beta) = -\frac{1}{n} \log L(\beta) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]. \quad (1.5)$$

Cuando la etiqueta real es $y_i = 1$, solo pesa el término $-\log p_i$, que crece sin cota cuando el modelo asigna a ese ejemplo una probabilidad p_i próxima a cero; cuando es $y_i = 0$, pesa $-\log(1 - p_i)$, con el castigo simétrico. A diferencia de la regresión lineal, cuya pérdida cuadrática tiene la solución cerrada $\hat{\beta} = (X^\top X)^{-1} X^\top y$, la entropía cruzada no se puede despejar: su gradiente es no lineal en β . Sí es convexa, con lo que tiene un único mínimo, y ese mínimo se alcanza donde el gradiente se anula. Usando la matriz de características $X \in \mathbb{R}^{n \times d}$, el vector de etiquetas y y el vector de predicciones $p = (p_1, \dots, p_n)^\top$, el gradiente tiene una forma compacta,

$$\nabla_\beta \mathcal{L}(\beta) = \frac{1}{n} X^\top (p - y), \quad (1.6)$$

el promedio de los errores $p_i - y_i$ ponderado por las características. Como no se anula en forma cerrada, se resuelve por **descenso de gradiente** [5]: se parte de unos coeficientes cualesquiera y se corrigen paso a paso en la dirección contraria al gradiente, $\beta \leftarrow \beta - \eta \nabla_\beta \mathcal{L}$, con $\eta > 0$ el tamaño de paso, hasta converger. Es el mismo método que está en la base de muchos algoritmos de aprendizaje. Para frenar el sobreajuste se añade un término de **regularización** que penaliza coeficientes grandes: la penalización L2 los encoge de forma suave, y la L1 lleva algunos a cero exacto, con lo que además descarta variables.

- **Ventajas:** interpretable por sus coeficientes, muy barata de entrenar y con probabilidades bien calibradas.
- **Desventajas:** solo traza fronteras lineales, por lo que resulta insuficiente cuando la separación entre clases es curva; conviene además eliminar antes las variables muy correlacionadas entre sí.

K vecinos más cercanos

El clasificador de k vecinos más cercanos (KNN) [8] no construye ningún modelo explícito: guarda los datos de entrenamiento y clasifica cada ejemplo nuevo por **vecindad**, según la clase de los ejemplos más próximos. La idea plantea dos preguntas: dónde se sitúan los datos y cómo se mide la cercanía. Lo primero se resuelve con el vector de características, que ubica cada ejemplo como un punto de $\mathbb{R}^d$. Para lo segundo se usa una distancia; las más habituales son la euclídea (la del teorema de Pitágoras) y la de Manhattan (la suma de diferencias en valor absoluto, como recorrer una cuadrícula de calles),

$$d_{\text{euclídea}}(p, q) = \sqrt{\sum_{i=1}^d (p_i - q_i)^2}, \quad d_{\text{Manhattan}}(p, q) = \sum_{i=1}^d |p_i - q_i|. \quad (1.7)$$

Ambas son casos de la distancia de Minkowski $(\sum_i |p_i - q_i|^r)^{1/r}$, con $r = 2$ y $r = 1$ respectivamente. Para clasificar un punto nuevo se calculan sus distancias a todos los ejemplos de entrenamiento, se toman los k más cercanos y se le asigna la clase mayoritaria entre ellos (Figura 1.6). La elección de k cambia el resultado: en la figura, entre los tres vecinos más próximos gana una clase por dos votos a uno, así que con $k = 3$ el punto se clasifica en ella; al ampliar el radio a $k = 5$ entran vecinos más lejanos de la otra clase que pueden invertir el voto. El valor de k es, por tanto, un hiperparámetro que hay que fijar con cuidado.

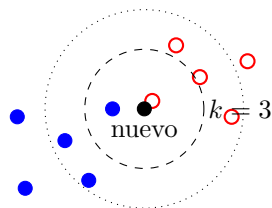


Figura 1.6: KNN: el punto nuevo (negro) se clasifica por la clase mayoritaria entre sus vecinos. El círculo interior es $k = 3$; ampliar el radio ($k = 5$) puede cambiar la decisión.

El parámetro k gobierna el compromiso sesgo-varianza. Un k pequeño hace la frontera muy sensible al ruido local: con $k = 1$ cada punto determina por sí solo su vecindad inmediata y basta un ejemplo mal etiquetado para crear una región aislada de clase equivocada, lo que sobreajusta. Un k grande promedia sobre vecinos cada vez más lejanos, suaviza la frontera y aumenta el sesgo, hasta el extremo de que con $k = N$ todo punto recibe la clase mayoritaria global. El mejor valor depende de los datos y se busca por validación, aunque una regla común de partida es $k = \sqrt{N}$, con N el número de ejemplos. Como todo se reduce a distancias, KNN es sensible a la escala de las variables y exige estandarizarlas antes. Sin ese paso, una característica medida en miles (por ejemplo, la longitud de la respuesta en caracteres) dominaría la distancia y anularía la contribución de otra acotada entre 0 y 1 (un solape), que apenas influiría por pequeña que fuese su diferencia; estandarizar deja a todas en el mismo rango y con el mismo peso a priori.

- **Ventajas:** no asume ninguna forma para la frontera entre clases, funciona bien con muchos datos y no requiere una fase de entrenamiento.
- **Desventajas:** clasificar es costoso, pues mide la distancia del punto nuevo a todo el entrenamiento, y el método se degrada cuando hay muchas dimensiones o la distancia está mal elegida.

Árboles de decisión

Un árbol de decisión [4, 19] clasifica imitando la forma en que una persona decide mediante preguntas encadenadas, cada una más fina que la anterior, y es además la pieza sobre la que se construyen los dos modelos siguientes. Tiene dos elementos: los *nodos* internos, donde se toma una decisión partiendo los datos por un umbral sobre una característica (“¿ $x_1 < u_1$?”), y las *hojas*, que asignan una clase (Figura 1.7). Clasificar un ejemplo consiste en recorrer el árbol desde la raíz, respondiendo en cada nodo la pregunta con los valores de ese ejemplo y siguiendo la rama correspondiente, hasta caer en una hoja, que da la clase predicha.

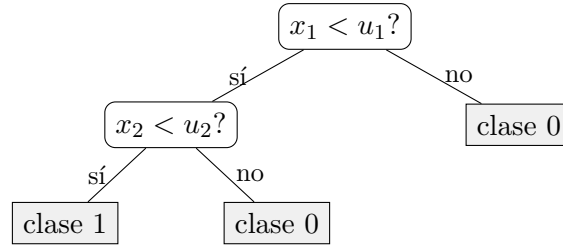


Figura 1.7: Árbol de decisión: los nodos parten los datos por un umbral; las hojas asignan la clase.

El árbol se construye de arriba abajo. Se parte de la raíz, que contiene todos los ejemplos, y se elige el corte que mejor separa las clases. Cada uno de los dos grupos resultantes se convierte en un nodo nuevo al que se aplica el mismo procedimiento, y este se repite de forma recursiva sobre los grupos que va generando hasta que dividir más no aporta mejora.

La pregunta en cada nodo es qué corte elegir de entre todos los posibles, uno por cada característica y cada umbral sobre ella. El criterio es la *pureza*: un buen corte deja a cada lado ejemplos de una sola clase. La razón es que una hoja predice la clase mayoritaria de los ejemplos que contiene, de modo que un grupo homogéneo se etiqueta sin error, mientras que uno donde las clases se reparten a partes iguales no permite una predicción fiable.

La *impureza* mide cuánto se mezclan las clases en un nodo. Las dos más usadas son el índice de Gini y la entropía.

Definición 1 (Impureza de un nodo). Sea un nodo con proporciones de clase $p_1, \dots, p_c$ (con $\sum_k p_k = 1$). Su *índice de Gini* y su *entropía* [21] son

$$G = 1 - \sum_{k=1}^c p_k^2, \quad H = - \sum_{k=1}^c p_k \log_2 p_k.$$

Ambas valen cero cuando el nodo es puro (una sola clase, alguna $p_k = 1$) y son máximas cuando las clases están repartidas por igual.

Las dos cuantifican la mezcla de clases, pero con lecturas distintas. El índice de Gini es la probabilidad de clasificar mal un ejemplo si se le asignara una clase al azar según las proporciones del nodo; la entropía mide la incertidumbre del nodo. En un problema de dos clases, ambas son máximas en el reparto mitad y mitad, donde el índice de Gini vale 0,5 y la entropía, 1.

La calidad de un corte se mide por cuánto baja la impureza al pasar del nodo padre a sus dos hijos. Si el padre tiene impureza $I(\text{padre})$ y el corte lo divide en un hijo izquierdo y uno derecho, que reciben una fracción w_{izq} y w_{der} de los ejemplos, la *reducción de impureza* del corte es

$$\Delta I = I(\text{padre}) - (w_{\text{izq}} I(\text{izq}) + w_{\text{der}} I(\text{der})), \quad (1.8)$$

esto es, la impureza del padre menos la impureza media de los hijos, en la que cada hijo pesa según la fracción de ejemplos que recibe. El árbol elige en cada nodo el corte que hace máxima esa reducción.

Lo ilustramos con un ejemplo. Un nodo con 40 ejemplos que alucinan y 60 limpios tiene proporciones 0,4 y 0,6, luego su índice de Gini es

$$G(\text{padre}) = 1 - 0,4^2 - 0,6^2 = 0,48.$$

Consideremos un corte que lo divide en un hijo izquierdo con 30 que alucinan y 10 limpias, y un hijo derecho con 10 que alucinan y 50 limpias. Cada hijo es más puro que el padre:

$$G(\text{izq}) = 0,375, \quad G(\text{der}) \approx 0,28.$$

El hijo izquierdo recibe 40 de los 100 ejemplos y el derecho los 60 restantes, así que la impureza media de los hijos es

$$\frac{40}{100} \cdot 0,375 + \frac{60}{100} \cdot 0,28 \approx 0,32,$$

y la reducción que aporta el corte es $\Delta G = 0,48 - 0,32 \approx 0,16$. El árbol compara así todos los cortes posibles y selecciona el de mayor reducción.

El procedimiento se repite en cada nodo nuevo y se detiene cuando el nodo ya es puro, cuando ningún corte mejora la pureza o cuando se alcanza un límite fijado de antemano, como la profundidad máxima.

Sin restricciones, el árbol crece hasta separar por completo el entrenamiento y lo memoriza, lo que lo hace muy flexible pero propenso al sobreajuste: un árbol profundo aprende las particularidades de los datos y falla con datos nuevos. La profundidad máxima y el número mínimo de ejemplos por hoja son los hiperparámetros que controlan ese crecimiento.

- **Ventajas:** captura interacciones entre variables, es insensible a la escala, maneja bien características irrelevantes y su lógica es legible como una cadena de reglas.
- **Desventajas:** un árbol único es inestable, pues pequeños cambios en los datos dan árboles muy distintos, y sobreajusta con facilidad. Los dos modelos siguientes corrigen esto combinando muchos árboles.

Random Forest

El bosque aleatorio (*Random Forest*) [3] combina muchos árboles para ganar la estabilidad de la que carece uno solo. Se apoya en una técnica llamada *bagging* (*bootstrap aggregating*): de la base de entrenamiento se extraen B muestras aleatorias con reemplazo, cada una del mismo tamaño que la original pero con ejemplos repetidos y otros ausentes, y con cada muestra se entrena un árbol distinto. Para clasificar un ejemplo nuevo, cada uno de los B árboles emite su voto y se toma la clase más votada,

$$\hat{y}(x) = \text{clase más votada entre } \{T_1(x), \dots, T_B(x)\}. \quad (1.9)$$

El bagging reduce la varianza sin aumentar el sesgo. Para ver por qué, consideramos la versión con predicciones numéricas, donde cada árbol devuelve la probabilidad de la clase positiva y el conjunto las promedia. Si cada árbol individual tiene varianza σ^2 y dos árboles distintos tienen una correlación ρ entre sus predicciones, la varianza del promedio de los B árboles es

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B T_b(x)\right) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2. \quad (1.10)$$

El segundo sumando se desvanece al crecer B , así que promediar muchos árboles cancela la parte no correlacionada del ruido. El primer sumando, en cambio, no depende de B : queda un suelo de varianza $\rho\sigma^2$ que solo baja si los árboles están poco correlacionados. Como el promedio no cambia la predicción esperada, el sesgo se mantiene. De aquí se sigue la estrategia del bosque: para reducir ese suelo de varianza hay que descorrelacionar los árboles. El bagging ya los diferencia al entrenar cada uno con una muestra distinta, y el bosque añade una segunda fuente de aleatoriedad: en cada corte no considera todas las características, sino un subconjunto aleatorio de ellas (típicamente $\sqrt{d}$ de las d disponibles en clasificación). Así ningún árbol se apoya siempre en la misma variable dominante, baja ρ y el conjunto gana estabilidad. El número de árboles B , su profundidad y cuántas características se consideran en cada corte son sus hiperparámetros, y suelen ajustarse por búsqueda.

- **Ventajas:** robusto y preciso, uno de los mejores clasificadores para datos tabulares; hereda del árbol la captura de interacciones y la insensibilidad a la escala, maneja muchas variables y apenas sobreajusta.
- **Desventajas:** pierde la interpretabilidad de un árbol único, es más costoso de entrenar y evaluar, y puede desajustarse con datos muy ruidosos si no se controla el número de árboles.

Gradient boosting

El *gradient boosting* [11] también combina árboles, pero de forma **secuencial** en vez de paralela. En lugar de promediar árboles independientes que votan a la vez, construye el modelo por etapas, de manera que cada árbol nuevo corrige los errores acumulados por los anteriores. Empieza con una predicción sencilla y, en cada etapa, añade un árbol ajustado para reducir ese error residual.

La clave del método es qué error corrige cada árbol nuevo. Tras $m - 1$ etapas el modelo F_{m-1} deja en cada ejemplo un *residuo*, la diferencia entre la etiqueta real y lo que predice. El *boosting* entrena el árbol siguiente para predecir ese residuo, de modo que sumarlo al modelo acerca la predicción a la etiqueta. En su forma general, el residuo que se corrige es el gradiente de la función de pérdida evaluado en las predicciones actuales, de ahí el nombre del método: cada árbol da un paso de descenso de gradiente, pero en el espacio de las funciones en vez de en el de unos coeficientes. El modelo tras m etapas es el anterior más el árbol nuevo,

$$F_m(x) = F_{m-1}(x) + \nu h_m(x), \quad (1.11)$$

donde h_m es el árbol de la etapa m , ajustado a los gradientes que deja F_{m-1} , y $\nu \in (0, 1]$ es la **tasa de aprendizaje** o *shrinkage*, que modera cuánto contribuye cada árbol. Reducir cada paso frena la corrección: una tasa pequeña avanza despacio y necesita más árboles, pero generaliza mejor, porque ningún árbol individual domina el resultado; una tasa grande, o demasiados árboles, sobreajustan. Tasa de aprendizaje y número de árboles se ajustan juntos, por ser las dos caras del mismo compromiso. A esa **regularización** por *shrinkage* se suman otras: limitar la profundidad de cada árbol para que corrija tendencias generales y no casos aislados, entrenar cada árbol sobre una submuestra aleatoria de los datos, y penalizar la complejidad del árbol en la propia función de pérdida.

La diferencia con el bosque aleatorio es de fondo. El bosque promedia árboles independientes para reducir la *varianza*, partiendo de árboles que ya de por sí aciertan; el boosting encadena árboles dependientes, cada uno corrigiendo lo que el anterior no logró, para reducir el *sesgo*, partiendo de un modelo débil que va mejorando. Eso suele dar mayor

precisión, a cambio de un ajuste más delicado y más lento. **XGBoost** [7] es una implementación eficiente y regularizada de esta idea, referencia habitual en problemas sobre datos tabulares.

- **Ventajas:** suele ofrecer la mayor precisión de los modelos clásicos sobre datos tabulares y maneja bien las interacciones entre variables.
- **Desventajas:** tiene más hiperparámetros que ajustar, entrena más despacio que un bosque y su riesgo de sobreajuste es mayor si no se regula con cuidado.

1.3.5. Valoración

La **valoración** cuantifica el rendimiento de los modelos ajustados sobre datos no empleados en el entrenamiento. Una *métrica* es un indicador de la calidad del modelo; todas las empleadas se construyen sobre el mismo recuento de aciertos y errores.

Tomando como clase positiva la que interesa detectar (la alucinación), cada predicción pertenece a una de estas cuatro categorías:

- **Verdaderos positivos (VP):** alucinaciones que el modelo marca como alucinación (acierto).
- **Verdaderos negativos (VN):** respuestas limpias que marca como limpias (acierto).
- **Falsos positivos (FP):** respuestas limpias que marca como alucinación (error de tipo I, una falsa alarma).
- **Falsos negativos (FN):** alucinaciones que marca como limpias (error de tipo II, una invención que pasa inadvertida).

Estos cuatro conteos se organizan en la *matriz de confusión* (Cuadro 1.1), que cruza la clase real de cada respuesta con la predicha por el modelo.

		Predicción del modelo	
		Alucinación	Limpia
Clase real	Alucinación	VP verdadero positivo (acierto)	FN falso negativo (error de tipo II)
	Limpia	FP falso positivo (error de tipo I)	VN verdadero negativo (acierto)

Tabla 1.1: Matriz de confusión. Cada respuesta cae en una de las cuatro casillas según su clase real (filas) y la predicción del modelo (columnas). Los aciertos (*VP* y *VN*) quedan en la diagonal; el falso negativo (error de tipo II) es una alucinación que pasa inadvertida y el falso positivo (error de tipo I), una falsa alarma sobre una respuesta limpia.

Los dos errores rara vez cuestan lo mismo. En un diagnóstico médico, un falso negativo deja sin tratar a un enfermo, mientras que un falso positivo trata a un sano. En la detección de alucinaciones el error caro es el falso negativo, una invención que llega al usuario sin control.

Precisión. De todas las respuestas que el modelo marca como alucinación, la fracción que lo son realmente. Mide la fiabilidad del modelo al señalar una alucinación: un valor cercano

a 1 indica que casi nunca se equivoca cuando predice un positivo.

$$\text{precisión} = \frac{VP}{VP + FP}. \quad (1.12)$$

Cobertura (también exhaustividad o sensibilidad). De todas las alucinaciones reales, la fracción que el modelo detecta. Un valor cercano a 1 indica que detecta casi todas.

$$\text{cobertura} = \frac{VP}{VP + FN}. \quad (1.13)$$

Especificidad. La cobertura de la clase negativa: de todas las respuestas limpias, la fracción que el modelo reconoce como tales. Interviene en las métricas que equilibran ambas clases.

$$\text{especificidad} = \frac{VN}{VN + FP}. \quad (1.14)$$

Precisión y cobertura varían en sentidos opuestos, y el umbral de decisión fija el equilibrio entre ambas. Si el modelo solo predice positivo cuando tiene alta confianza, acierta casi siempre que lo hace (precisión alta) pero omite los positivos dudosos (cobertura baja); si predice positivo con poca evidencia, apenas omite ninguno (cobertura alta) a costa de numerosas falsas alarmas (precisión baja). Las métricas siguientes resumen ese equilibrio en un solo número.

Valor F_1 . Combina precisión y cobertura en su *media armónica*, que solo es alta cuando ambas lo son; si una se acerca a cero, el valor de F_1 también.

$$F_1 = 2 \frac{\text{precisión} \cdot \text{cobertura}}{\text{precisión} + \text{cobertura}}. \quad (1.15)$$

Se prefiere la media armónica a la aritmética porque esta valdría 0,5 para un modelo con precisión 1 y cobertura 0, que no detecta nada.

Valor F_β . El F_1 da igual peso a precisión y cobertura. Su generalización F_β pondera la cobertura β^2 veces más que la precisión; el caso $\beta = 2$ (F_2) lo pesa cuatro veces más, útil cuando el falso negativo es el error caro.

$$F_\beta = (1 + \beta^2) \frac{\text{precisión} \cdot \text{cobertura}}{\beta^2 \text{precisión} + \text{cobertura}}. \quad (1.16)$$

Accuracy. La fracción de aciertos sobre el total. Es intuitiva, pero resulta engañosa cuando una clase domina: si el 90 % de los ejemplos son negativos, predecir siempre negativo ya da un 90 % de accuracy sin detectar un solo positivo.

$$\text{accuracy} = \frac{VP + VN}{VP + VN + FP + FN}. \quad (1.17)$$

Balanced accuracy. Corrige esa distorsión promediando la tasa de aciertos de cada clase por separado: la cobertura en la clase positiva y la especificidad en la negativa. Así la clase minoritaria pesa igual que la mayoritaria.

$$\text{balanced accuracy} = \frac{1}{2} (\text{cobertura} + \text{especificidad}). \quad (1.18)$$

Un ejemplo numérico ilustra las definiciones anteriores. Sobre 300 respuestas con 100 alucinaciones reales, un detector acierta 70 ($VP = 70$, $FN = 30$) a costa de 20 falsas alarmas ($FP = 20$, $VN = 180$). Entonces

$$\text{precisión} = \frac{70}{90} \approx 0,78, \quad \text{cobertura} = \frac{70}{100} = 0,70, \quad F_1 \approx 0,74, \quad \text{accuracy} = \frac{250}{300} \approx 0,83.$$

La accuracy (0,83) resulta alta porque el modelo acierta casi todos los negativos, que son mayoría; la balanced accuracy, $\frac{1}{2}(0,70 + 0,90) = 0,80$, corrige esa cifra al dar el mismo peso a las dos clases.

Todas las métricas anteriores dependen del umbral con que la probabilidad se convierte en decisión. Dos curvas recorren todos los umbrales a la vez.

Curva ROC y AUC. La curva ROC contrapone la tasa de verdaderos positivos (la cobertura) frente a la tasa de falsos positivos al variar el umbral [10]. Su área bajo la curva, el **AUC**, resume en una sola cifra el rendimiento en todos los umbrales.

Teorema 1 (Área bajo la curva ROC). Sea $s(x)$ la puntuación que el clasificador asigna a un ejemplo x , y sean X^+ y X^- un positivo y un negativo tomados al azar de forma independiente. El área bajo la curva ROC coincide con la probabilidad de que el positivo puntúe por encima del negativo:

$$\text{AUC} = \int_0^1 \text{TVP}(\text{TFP}) d(\text{TFP}) = \mathbb{P}(s(X^+) > s(X^-)),$$

con el convenio de contar como $\frac{1}{2}$ los empates $s(X^+) = s(X^-)$.

La primera igualdad es la lectura geométrica; la segunda, la probabilística [15]. El AUC tiene dos propiedades que lo hacen sólido para comparar modelos:

- es invariante a la escala del score, pues solo importa el orden de las predicciones;
- es invariante al umbral, pues los resume todos, así que compara sin fijar un punto de operación.

Curva precisión–cobertura. Contrapone precisión frente a cobertura al variar el umbral. A diferencia del AUC, sí refleja el desbalance, porque la precisión depende de la proporción de positivos; por ello resulta más informativa que el AUC cuando la clase positiva es minoritaria. Un buen modelo mantiene la precisión alta aun a cobertura alta, y su curva se acerca a la esquina superior derecha.

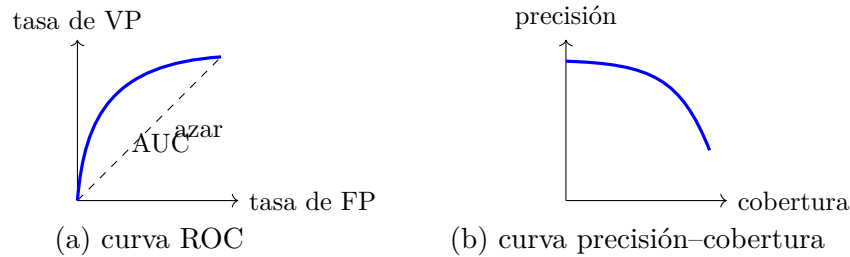


Figura 1.8: Las dos curvas que recorren todos los umbrales. La ROC resume la separación en el AUC; la precisión–cobertura muestra a partir de qué cobertura se degrada la precisión.

Índice de Youden. Fijado el modelo, resta determinar el umbral que convierte su *score* en decisión. El índice de Youden selecciona el que maximiza

$$J = \text{sensibilidad} + \text{especificidad} - 1, \quad (1.19)$$

esto es, el punto de la curva ROC más alejado de la diagonal, donde mejor se separan las dos clases [25]. Rebajar el umbral aumenta la cobertura a costa de la precisión, y elevarlo produce el efecto contrario; el umbral es, por tanto, junto al remuestreo, una palanca frente al desbalance.

Bibliografía

- [1] Charu C. Aggarwal and ChengXiang Zhai. *Mining Text Data*. Springer, New York, 2012.
- [2] Ana Azevedo and Manuel Filipe Santos. KDD, SEMMA and CRISP-DM: A parallel overview. In *Proceedings of the IADIS European Conference on Data Mining*, pages 182–185, Amsterdam, 2008.
- [3] Leo Breiman. Random forests. *Machine Learning*, 45(1), 2001.
- [4] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth, Monterey, CA, 1984.
- [5] Augustin-Louis Cauchy. Méthode générale pour la résolution des systèmes d’équations simultanées. *Comptes Rendus Hebdomadaires des Séances de l’Académie des Sciences*, 25:536–538, 1847.
- [6] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 2002.
- [7] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [8] Thomas M. Cover and Peter E. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 1967.
- [9] David R. Cox. The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B*, 20(2), 1958.
- [10] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 2006.
- [11] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 2001.
- [12] Stuart Geman, Elie Bienenstock, and René Doursat. Neural networks and the bias/variance dilemma. *Neural Computation*, 4(1):1–58, 1992.
- [13] Isabelle Guyon and André Elisseeff. An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3, 2003.

- [14] Isabelle Guyon, Jason Weston, Stephen Barnhill, and Vladimir Vapnik. Gene selection for cancer classification using support vector machines. *Machine Learning*, 46(1–3), 2002.
- [15] James A. Hanley and Barbara J. McNeil. The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1):29–36, 1982.
- [16] Zellig S. Harris. Distributional structure. *WORD*, 10(2–3):146–162, 1954.
- [17] Roderick J. A. Little and Donald B. Rubin. *Statistical Analysis with Missing Data*. John Wiley & Sons, Hoboken, NJ, 3rd edition, 2019.
- [18] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, New York, 1997.
- [19] J. Ross Quinlan. Induction of decision trees. *Machine Learning*, 1(1):81–106, 1986.
- [20] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 1988.
- [21] Claude E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948.
- [22] Mervyn Stone. Cross-validatory choice and assessment of statistical predictions. *Journal of the Royal Statistical Society: Series B (Methodological)*, 36(2):111–147, 1974.
- [23] John W. Tukey. *Exploratory Data Analysis*. Addison-Wesley, Reading, MA, 1977.
- [24] David H. Wolpert. The lack of a priori distinctions between learning algorithms. *Neural Computation*, 8(7):1341–1390, 1996.
- [25] William J. Youden. Index for rating diagnostic tests. *Cancer*, 3(1):32–35, 1950.